Deterministic Optimization

Discrete Optimization: Modeling w/ binary variables

Shabbir Ahmed
Anderson-Interface Chair and Professor
School of Industrial and Systems Engineering

Modeling Exercises 2

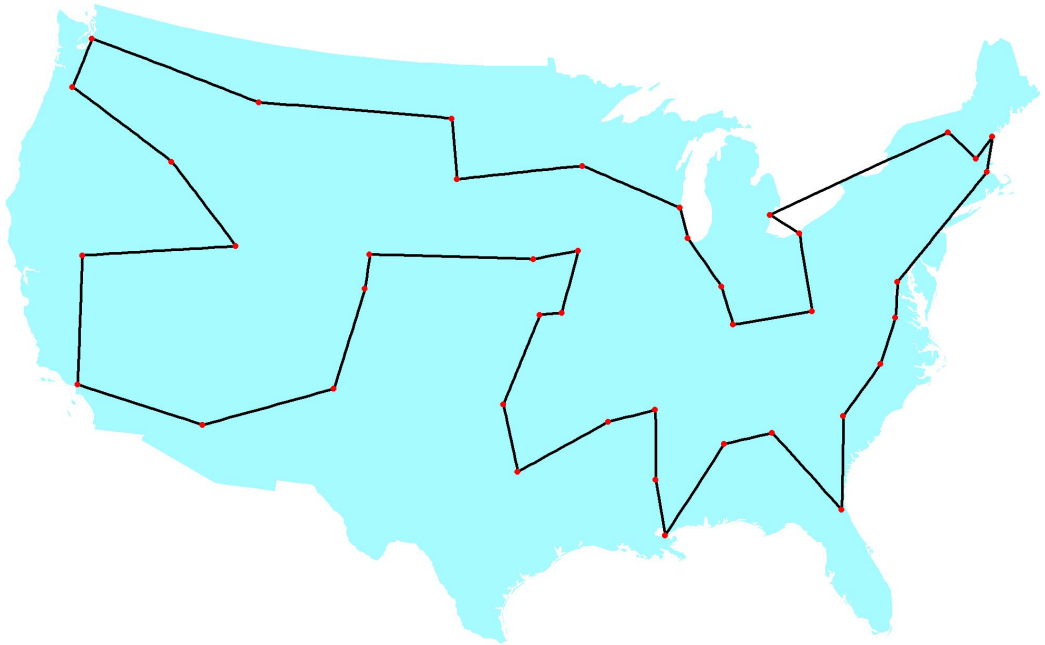
Modeling Exercises 2

Learning Objectives

- Observe modeling the traveling salesman problem (TSP)
- Solve a TSP example with PuLP and Python

Traveling Salesman Problem (TSP)

Given a collection of n cities, along with pair-wise distances between them, find the shortest way to visit all cities are return to the starting point.



A tour through 42 US Cities
Source: http://www.math.uwaterloo.ca/tsp

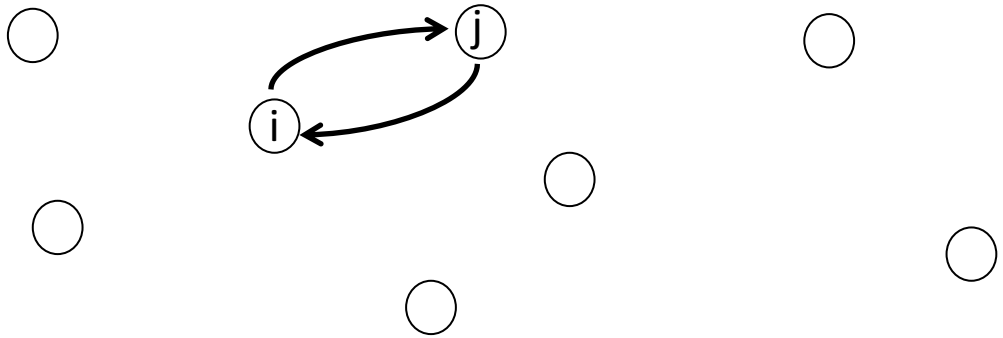
Formulation

Variables (for $i \neq j$):

$$x_{ij} = \begin{cases} 1 & \text{if city } i \text{ is visited immediately before city } j \\ 0 & \text{otherwise} \end{cases}$$

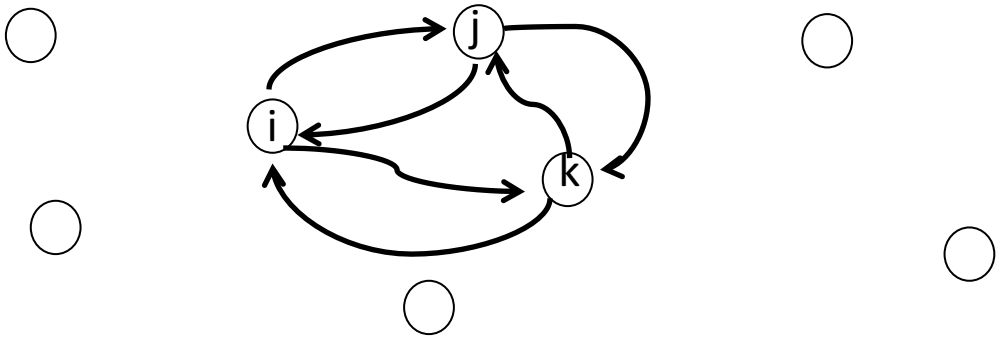
$$\begin{aligned}
\min \quad & \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{j=1}^{n} x_{ij} = 1 && \forall\ i = 1, \ldots, n \\
& \sum_{j=1}^{n} x_{ji} = 1 && \forall\ i = 1, \ldots, n \\
& \sum_{i \in S} \sum_{j \in S} x_{ij} \leq |S| - 1 && \forall\ S \subset \{1, \ldots, n\}, |S| \geq 2
\end{aligned}$$

Formulation: Subtour Elimination



$$S = \{i, j\}$$
$$x_{ij} + x_{ji} \leq 1$$

Formulation: Subtour Elimination



$$S = \{i, j, k\}$$

$$x_{ij} + x_{ik} + x_{ji} + x_{jk} + x_{ki} + x_{kj} \leq 2$$

Formulation: Subtour Elimination

- There are many (exponential in n) subtour elimination constraints
- Solve without any subtour elimination constraints
- Inspect the solution to check if there are subtours
- Add the subtour elimination constraint corresponding to the found subtour
- Repeat

PuLP Code

```
# Solving a TSP Example with PuLP
# Install PuLP and glpk before running this code
# http://pythonhosted.org/PuLP/
# http://www.gnu.org/software/glpk/
# This example is adapted from http://pymprog.sourceforge.net/subtour.html

import pulp as p

# Distance matrix
C = [
[0,86,49,47,90,90,50],
[86, 0,68,79,93,24, 5],
[49,68, 0,16, 7,72,67],
[57,79,16, 0,90,69, 1],
[31,93, 7,90, 0,86,59],
[69,24,72,69,86, 0,81],
[50, 5,67, 1,2,81, 0]
]

# Number of cities
n = len(C)

# Set of cities
N = range(n)

# Edges = (ij) for i!= j
E =[(i,j) for i in N for j in N if i!=j]

```

PuLP Code

```
29 # Set up initial PuLP model without subtour elim constraints
30 m = p.LpProblem("TSP",p.LpMinimize)
31
32 # Create variables
33 x = p.LpVariable.dicts('x',E,cat=p.LpBinary)
34
35 # Create objective
36 m += sum([C[i][j]*x[(i,j)] for (i,j) in E]), "Objective"
37
38 # Add constraints
39 for i in N:
40     m += sum([x[(i,j)] for j in N if i!=j ]) == 1, "One out of "+str(i)
41     m += sum([x[(j,i)] for j in N if i!=j ]) == 1, "One in to "+str(i)
42
43 # Solve model
44 m.solve()
45
46 # Collect objective and solution
47 vobj = p.value(m.objective)
48 xsol = {e: p.value(x[e]) for e in E}
49
```

PuLP Code

```
50 # Find a subtour starting from city 0
51 # We only search for these kinds of subtours
52 def subtour(xsol):
53     succ = 0
54     subt = [succ] #start from city 0
55     while True:
56         succ=sum(xsol[succ,j]*j for j in N if j!=succ)
57         if succ == 0: break #tour found
58         subt.append(int(succ))
59     return subt
60
```

PuLP Code

```
62 # Add subtour elimination constraints and repeat
63 while True:
64     subt = subtour(xsol)
65     if len(subt) == n:
66         print("Optimal tour found: %r" %subt)
67         print("Optimal tour length: %g"%vobj)
68         break
69     print("Subtour found: %r"% subt)
70 
71     #now add a subtour elimination constraint
72     m+= sum([x[(i,j)] for i in subt for j in subt if i!=j]) <= len(subt)-1
73 
74     # solve model
75     m.solve()
76     vobj = p.value(m.objective)
77     xsol = {e: p.value(x[e]) for e in E}
78 
```

PuLP Code Output

```
Subtour found: [0, 2, 4]
Subtour found: [0, 2, 3, 6, 4]
Subtour found: [0, 3, 6, 1, 5]
Optimal tour found: [0, 5, 1, 6, 3, 2, 4]
Optimal tour length: 174
```

Summary

- We studied a formulation of the TSP and solved it using PuLP
- There are alternative formulations